

杂项板子

生成时间: 2026-06-01

默认省略通用 C++ 头文件与空壳 main

目录

- 1 动态规划 3
 - 1.1 动态规划/对 dp 的一些思考 3
 - 1.2 动态规划/dp 笔记/数位 dp 笔记 3
 - 1.3 动态规划/dp 笔记/普通 dp 常见状态 3
 - 1.4 动态规划/dp 笔记/状压 dp 常见状态 4
- 2 数据结构 5
 - 2.1 数据结构/数据结构笔记/数据结构 trick 5
- 3 图论 6
 - 3.1 图论/Trick/图相关 trick 6
 - 3.2 图论/图论笔记/图论笔记(几类特殊图) 6
- 4 博弈论 10
 - 4.1 博弈论/nim 游戏 SG 函数 10
- 5 通用 Trick 与杂项 13
 - 5.1 其他/Trick/杂项相关 trick 13
- 6 字符串 15
 - 6.1 字符串/笔记/一些比较神秘的 hash 手法 15
 - 6.2 字符串/笔记/字符串 trick 15

1 动态规划

1.1 动态规划/对 dp 的一些思考

有时候想想 这个状态是否完备？ 状态数是不是可以通过某些手段缩减？

多位 dp 的时候能把小的项放在前面 会快一点(?)

状压子集枚举 3^n st 的子集 $\text{for}(\text{int } i=\text{st}; i=(i-1)\&\text{st})$ st 的超集 $\text{for}(\text{int } i=\text{st}; i<(1<<n); i=(i+1)|\text{st})$

退背包 一般用于我要统计不包含一个物品的方案数 就是反着做背包就行 ->

```
auto add=[&](int cnt,int siz,int val)->void{
    for(int i=cnt;i>=1;i--){
        for(int j=siz;j>=val;j--){
            dp[i][j]+=dp[i-1][j-val];
        }
    }
};
auto del=[&](int cnt,int siz,int val)->void{
    for(int i=1;i<=cnt;i++){
        for(int j=val;j<=siz;j++){
            dp[i][j]-=dp[i-1][j-val];
        }
    }
};
```

前后缀背包？

博弈论 dp 我们可以仿照 sg 函数的感觉进行状态设计 $\text{dp}[i]=1$ 表示先手必胜 $\text{dp}[i]=0$ 表示先手必败 重要是先后手之间的传递

概率/期望 dp 最重要的是写出转移式子,如果出现了自指的情况,可以移项

1.2 动态规划/dp 笔记/数位 dp 笔记

以下为记忆化搜索函数 **dfs** 的常设定的形参

- pos: **int** 型变量, 表示当前枚举的位置, 一般从高到低
- limit: **bool** 型变量, 表示枚举的第 pos 位是否受到限制
 - ▶ 为 **true** 表示取的数不能大于 $a[\text{pos}]$, 而只有在 $[\text{pos}+1, \text{len}]$ 的位置上填写的数都等于 $a[i]$ 时该位才为 **true**
 - ▶ 否则表示当前位没有限制, 可以取到 $[0, R-1]$, 因为 R 进制的数中数位最多能取到的就是 $R-1$
- last: **int** 型变量, 表示上一位 (第 $\text{pos}+1$ 位) 填写的值
 - ▶ 往往用于约束相邻数位之间的关系的问题
- lead0: **bool** 型变量, 表示是否有前导零, 即在 $\text{len} \rightarrow (\text{pos}+1)$ 这些位置是不是都是前导零
 - ▶ 基于常识, 我们往往默认一个数没有前导零, 也就是最高位不能为 0, 即不会写为 **000123**, 而是写为 **123**
 - ▶ 只有没有前导零的时候, 才能计算贡献
 - ▶ 那么前导零可用时答案有关?
 - 统计 0 出现次数
 - 相邻数位的差值
 - 以后问题为起点确定的奇偶位
- sum: **int** 型变量, 表示当前 $\text{len} \rightarrow (\text{pos}+1)$ 的数位和
- r: **int** 型变量, 表示整数 x 取模某个数 m 的余数
 - ▶ 该参数一般会固定在: 约束中出现了 “能被 m 整除”
 - ▶ 当然也可以拓展成取数权取模的结果
- st: **int** 型变量, 用于状态压缩
 - ▶ 对一个集合的数位权上的出现次数的奇偶性有要求时, 其二进制形式就可以表示每个数出现的奇偶性

1.3 动态规划/dp 笔记/普通 dp 常见状态

有些 dp 状态可以奇偶项不同 (

$\text{dp}[\text{len}][j][k]$ 表示长度 len 满足尾巴是 $j, k, j < k$ 的序列数 类似的记录尾部状态的 dp 有很多

$\text{dp}[i][j]$ 表示将整数 j 划分为 i 个不同的正整数的方案数 $\Leftrightarrow$ 等效命题: 存在划分 $l_1, l_2, l_3, l_4 \dots l_k$ 满足 $l_1 + l_2 + l_3 + l_4 + \dots + l_k = j, l_1 < l_2 < l_3 < \dots$ 两者是双射 转移 考虑一个 j 的划分 $\{l_1, l_2, l_3 \dots l_i\}$ case1> 划分中不存在 1 $\text{dp}[i][j] <= \text{dp}[i][j-i]$ 整体-1 case2> 划分中存在 1 $\text{dp}[i][j] <= \text{dp}[i-1][j-i]$ 整体-1,且划分中 1 的个数-1

$\text{dp}[i][k]$ 表示合法的数量为 i 的前缀某排列子集末尾的 rk 为这个前缀排列中 rk=k 的方案数 ->解决某类值域上的相邻约束排列计数问题 一类计数问题, 我们可以不考虑放具体的数, 而是考虑相对位置, 这 and 具体数是一个双射关系。 https://atcoder.jp/contests/abc457/tasks/abc457_f

树上背包经典状态 $\text{dp}[u][i]$ 表示以 u 为根的子树中, 背包大小为 i 的... 复杂度是经典的 on^2 为啥??? 因为背包合并本质笛卡尔积 每一对节点 (x, y) 只在 $\text{lca}(x, y)$ 处被计算一次, 所以复杂度是 $O(n^2)$ 的

一类异或=0的dp $dp[i]$ 表示元素异或=0的方案数 可以钦定 $i=x$ 利用容斥来转移 具体来说 考虑 $dp[i-2]$ $dp[i-1]$ 的含义 然后再考虑组合意义

一类路径选址的dp(k 聚类, IOI 1996 邮局问题) 通常我们先考虑一个区间的代价, 再按划分 dp/连续段 dp 的套路去做

1.4 动态规划/dp 笔记/状压 dp 常见状态

$dp[st][i]$ 表示经过图中某些点构成的集合为 st , 且最后一个点是 i 的某性质

2 数据结构

2.1 数据结构/数据结构笔记/数据结构 trick

线段树可以维护的带结合律的操作 摩尔投票，动态树直径，区间等差数列加，区间最大子段和，ddp 摩尔投票维护绝对众数（区间内出现次数严格大于一半的数）：维护每个节点的候选众数和净剩票数，合并时攻击抵消或合并，注意验证是否为绝对众数，可以维护下标然后二分。动态树直径：考虑点作为原树的点，区间作为覆盖点的直径，考虑两个直径的合并 $(u,v),(x,y)$ ，新的直径只可能是 $(u,v),(x,y),(u,x),(v,y),(u,y),(v,x)$ 这六种情况，合并时枚举这六种情况取最大值即可。区间等差数列加：节点维护 sum, 首项，公差，lazytag 维护首项和公差，tag 合并直接加，而下放直接修正偏移量就行 如果没有单点修/单点查，可以用二阶差分，考虑 $d1[i]=a[i]-a[i-1], d2[i]=d1[i]-d1[i-1]$ ，首项 k，公差 d 修改是： $d2[l]+=k, d2[l+1]+=-d-k, d2[r+1]-=k+(r-l+1)d, d2[r+2]-=k+(r-l)d$ 区间最大子段和：维护四个信息：区间的最大后缀和，区间的最大前缀和，区间和，区间最大子段和

查询一个区间是否包含一个区间，这个事情是可以二维数点做的，具体来说，把询问离线，按 r 排序，用 bit 维护 l 的位置就行，在线可以用主席树维护。

SWAG 用来解决这样的问题 维护一个满足结合律的元素的窗口。即 窗口内是结合的元素 例如 on 来做这样一个问题 固定 k 的窗口 查询所有窗口的 max 的 xor 或者 查询最长的满足某个矩阵乘法有某类特征的字序列

滑动窗口维护定长区间最大值：单调队列

3 图论

3.1 图论/Trick/图相关 trick

树上任意一点 v ，距离其最远的点一定是该树某条直径的两个端点之一。考虑反证法即可

联通块 <https://ac.nowcoder.com/acm/contest/view-submission?submissionId=79644138&returnHomeType=1&uid=719203876>

有时候想想菊花图呢

每个点只有一个出边是内向基环树，可以倍增 每个点只有一个入边是外向基环树

最小瓶颈路：一个点到另一个点所有路径中最大点权的最小值，可以用 $dijk$ 算 多次询问的话可以求出 mst 然后跑树剖/rmq

以 u 为根的子树拓扑序列的计数是 $(n-1)! / \text{所有子树大小的乘积 (除 } u)$

一个有向树的存在 $i \rightarrow j$ 边，则他的传递闭包中 (到达 j 的集合) 交 (i 可达的集合).size() ≥ 2 ; 如果按传递闭包的 sz 排序，这个是天然符合拓扑序的。

一个无向树的传递闭包，按行按集合划分的等价类是描述同链的

欧拉图：存在一条欧拉回路，能经过每个边一次，且起终点都是同一个点 半欧拉图：存在一条欧拉路径，能经过每个边一次

无向图是欧拉图的充要：所有点的度数都是偶数 无向图是半欧拉图的充要：恰好有 2 个顶点的度数是奇数，其余所有顶点的度数都是偶数

有向图是欧拉图的充要：所有点的出度等于入度 有向图是半欧拉图的充要：一个点的出度-入度=1（起点），另一个点的入度-出度=1（终点），其余所有点的出度等于入度

注意上述的判定都是基图全联通的情况，即忽略边的方向构成的图。

3.2 图论/图论笔记/图论笔记(几类特殊图)

3.2.0.0.1 记几类特殊图

被傻逼 abc e 题击败了 遂记录之

3.2.0.0.1.1 0.一些概念

1).团：一个完全子图 2).最大独立集：一个点数最多的顶点集，其中任意两个顶点之间都没有边 >可以发现这两是对偶关系

3).最小染色：用最少的颜色给点染色使得所有边连接的两点颜色不同。 4).色数：最小染色中使用的颜色数 >其实下面所述都属于完美图，即最大团=最小染色数

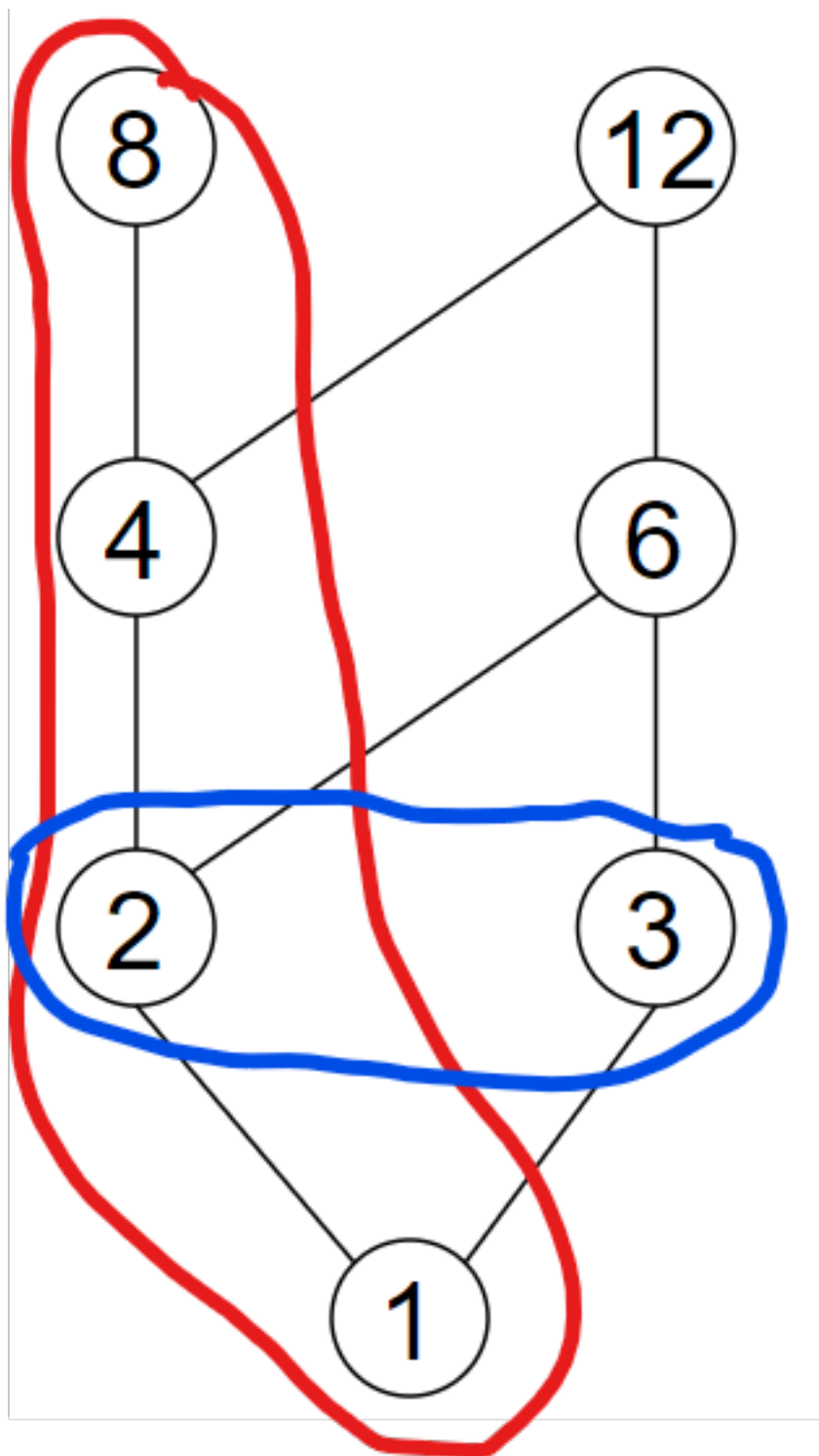
3.2.0.0.1.2 1.可比图->逆序图

可比图：本质 Hasse 图，刻画在偏序关系上的图 两元素之间可比，则存在一条无向边 >通常会画成有向图（如果定义了偏序关系），但本质是无向的（只关心可比关系）

链：构成一条链的元素两两之间可比 $\Rightarrow$ 对应图中的 团。反链：构成一条反链的元素两两之间不可比 $\Rightarrow$ 对应图中的 独立集。链划分：将一个偏序集划分成若干条链

Dilworth 定理：最小链覆盖数 = 最大反链的大小 = 最大独立集的大小 对偶的 Dilworth 定理：最小反链覆盖数 = 最长链的大小 = 最大团的大小

其实此处的最长链是好求的，就是 DAG 的最长路 最大反链也是好求的，跑个传递闭包，按二分图的方法跑最小链覆盖就行



>这是个整除图来着，红色的是链 蓝色的是最大反

链 注意到最小链划分数是 2，最大反链的元素数目也是 2

逆序图：给定一个序列，逆序图是表示序列中元素之间逆序关系的图 具体来说，对于序列中的元素 $a[i]$ 和 $a[j]$ ，如果 $i < j$ 且 $a[i] > a[j]$ ，则存在一条从 i 到 j 的无向边 我们把逆序图套到可比图上，发现，逆序图的最长反链（最大独立集）就是最长上升子序列 因为最长上升子序列中任意两个元素之间都不存在逆序关系，即两两不可比 同理我们把这个序列划分成最少的下降子序列，会发现，这个个数正好是最长上升子序列的长度 即 Dilworth 定理：最小链覆盖数 = 最大反链的大小

同理，逆序图的最长链（最大团）就是最长下降子序列，其中任意两个元素之间都存在逆序关系，即两两可比 同理我们把这个序列划分成最少的上升子序列，会发现，这个个数正好是最长下降子序列的长度 即对偶的 Dilworth 定理：最小反链覆盖数 = 最长链的大小

所以我们看到可比图的时候，先想想这个可比图的链是啥含义，反链又是啥含义，这样有可能把一般可比图的求最大反链的 $O(n^{1/2} * m)$ 优化成 $O(n \log n)$

3.2.0.0.1.3 2.二分图

详细证明见二分图最大匹配.cpp 注释部分

二分图最大匹配 最大流 $O(n^1/2 * m)$

二分图最小点覆盖：在一张无向图中选择最少的顶点，满足每条边至少有一个端点被选 二分图中，最小点覆盖中的顶点数量等于最大匹配中的边数量。(Kőnig 定理)

二分图最大独立集：在一张无向图中选择最多的顶点，满足两两之间互不相邻。二分图中，最大独立集中的顶点数量等于 n -最小点覆盖中的顶点数量 > 其实对于一般图来说，最大独立集也 = n -最小点覆盖中的顶点数量

有向无环图最小路径覆盖：在一张有向图中，选择最少数量的简单路径，使得所有顶点都恰好出现在一条路径中。->有向无环图的最小路径覆盖数等于顶点数减去最大匹配数 通过 dag 构造的二分图如下：将每个顶点拆成两个顶点， v_{in} v_{out} 对于原图中的每条有向边 $u \rightarrow v$ ，在二分图中连边 $v_{in}-u_{out}$

也就是说 对于一般的 dag 我们可以通过这个转换 最大流 $O(n^1/2 * m)$ 求出最小链划分 注意路径是不交的 如果要相交先对原图跑一个传递闭包

3.2.0.0.1.4 3.区间图/一堆区间的简单问题

区间图：给定 n 个区间，区间图是表示区间之间包含关系的图 区间有交集则连边

注意到区间图没有长度 ≥ 4 的无弦环，所以他是弦图 随便画画图就能证明

3.2.0.0.1.4.1 1).最大团：直线上重叠层数最多的位置 <-> 最小染色:给每个区间染色使得重叠区间颜色不同，最少颜色数

此处的最大团=最小染色比较显然

经典模型：求区间图最大团 >把区间变成 $[l,+1],[r,-1]$,然后对坐标排序,注意相同的时候+1 放前面即可，做扫描线即可

经典模型：求区间最小染色的方案 >建立小根堆，里面存{某颜色的结束时间, 颜色 ID}，每次取出堆顶，如果堆顶的结束时间小于当前区间的开始时间，则该区间可以染该颜色，否则该区间无法染色，需要新开一个颜色，然后加入堆中

3.2.0.0.1.4.2 2).最大独立集：求最多的不交区间数

这是经典的，按右端点 R 从小到大排序，选了 i 后，下一个选 $L_j > R_i$ 的第一个 j 。

当然当我们要求最小点覆盖的时候只要用 n 减去最大独立集即可

3.2.0.0.1.4.3 3).选最少的点，使得每个区间内至少包含一个点。

经典贪心：按右端点 R 从小到大排序。如果当前区间不交，贪心地选该区间的最右端点（因为它能尽可能多地向右覆盖后续区间的左端点）。

当然其实我们知道 2 和 3 是等价的，你可以发现他们的本质是一样的

3.2.0.0.1.4.4 4).别的附赠小甜品

给定大区间 $[S, T]$ 和 N 个小区间，求最少选几个小区间能完全覆盖 $[S, T]$

按 l 排序，维护当前已覆盖到的右边界 r ，在当前区间中找最大的 r 即可

每个区间 $[L_i, R_i]$ 有权值 W_i ，求互不相交的区间集合使得权值和最大 >随便 dp 即可

3.2.0.0.1.5 4.弦图

弦图：没有大于 3 的“无弦环”的图，即他没有大洞 >树也是弦图！

3.2.0.0.1.5.1 1).最大势算法(mcs)生成完美消除序列(peo)

完美消除序列：序列 $v_1, \dots, v_n$ 中，对于任意 v_i ，其在序列后方 $(v_{i+1} \dots v_n)$ 的所有邻居构成一个团（即两两相连）。

给图中的每个点分配一个编号， n 倒数到 1 进行编号。贪心策略：每次从未编号的节点中，选择与“已编号节点”相连边数最多的那个点进行编号。>感性理解一下，每次选相连边数最多的那个点可以保证团的纯度

我们需要维护一个 $label[i]$ ：表示节点 i 目前与多少个已编号的节点相连

朴素实现是 $O(n^2)$ 的，不够优秀，我们可以用一个懒更新的手法把他变成 $O(n + m)$ ，具体来说，我们用一个桶 $st[i]$ 存储 $label = i$ 的点，然后维护一个 $mxlab$ 指向桶的末尾，再开个 $vis[i]$ 表示 i 是否被编号过，这样我们每次在取的时候可以检查 vis 数组，不成立的废弃即可。

注意到，边 (u, v) 最多贡献一次 $label$ 值， st 的 $push$ 次数不超过 m ， $mxlab$ 的回退总量 $\leq n$ (一个点的邻接点数 $\leq n - 1$)，所以复杂度是 $O(n + m)$ 的。

3.2.0.0.1.5.2 2).验证完美生成序列

考虑优化一下 PEO 的定义，把他写成归纳的形式：对于当前点 u ，我们不需要检查其后方邻居是否“两两”有边。我们只需要找到 u 在序列后方位位置最靠前的那个邻居，记为 p 。只要满足： u 的所有其他后方邻居，都是 p 的邻居，那么 u 就满足性质。>随便归纳一下就行，类似于 n 循环比赛

考虑实现

用 rk 反映射 peo 序列， $rk[i]$ 表示 i 在 peo 中的位置，我们处理出每一个 p 的检查列表 $chkls[p]$ ，然后按 peo 序列的顺序（其实这个是不必要的，按 $1-n$ 都可以），遍历 p ，给 p 的邻居打上时间戳，然后在检查列表里的点检查邻居即可。

注意到每个 u 只属于一个 p ，所以时间复杂度是 $O(n + m)$ 的。

3.2.0.0.1.5.3 3).求解最大团，色数和最大独立集

根据完美图性质，最大团=最小染色数。这个最大团已经在生成 PEO 的过程中算出来了。回忆 $label[i]$ ：表示节点 i 目前与多少个已编号的节点相连。根据弦图性质，这 $label[u]$ 个邻居加上 u 自己，恰好构成了一个大小为 $label[u] + 1$ 的团。

最大独立集，贪心着求，开 vis 数组，按 PEO 选尽量前面即可。>感性理解，考虑交换论证，设 u, v, u 的出现早于 v ， u 的团包含 v ，考虑选 u ，团内只能选一个，不劣，考虑选 v ，扼杀了 v 构成的团内的某些可能，劣。

4 博弈论

4.1 博弈论/nim 游戏 SG 函数

4.1.1 尼姆 (Nim) 游戏

Nim 游戏的规则是这样的：地上有 n 堆石子，甲、乙两人交替取石子。每人每次只能从任意一堆石子里面取，至少取 1 枚，不能不取。最后没有子可取的人就输了。假如甲是先手，且已知每堆石子的数量 a_i ，问是否存在先手必胜的策略。

4.1.1.1 结论

- 若初态为必胜态 ($a_1 \wedge a_2 \wedge \dots \wedge a_n \neq 0$)，则先手必胜；
- 若初态为必败态 ($a_1 \wedge a_2 \wedge \dots \wedge a_n = 0$)，则先手必败。

4.1.1.2 定理证明

4.1.1.2.1 定理 1：必胜态的后继状态至少存在一个必败态

证明：

设 $a_1 \wedge \dots \wedge a_n = s \neq 0$ ，设 s 的二进制位为 1 的最高位是第 k 位，则 $a_1, \dots, a_n$ 中一定有奇数个 a_i 的二进制的第 k 位为 1。用 $a_i \wedge s$ 去替换 a_i ，则：

$$a_1 \wedge \dots \wedge (a_i \wedge s) \wedge \dots \wedge a_n = a_1 \wedge \dots \wedge a_i \wedge s \wedge \dots \wedge a_n = s \wedge s = 0$$

这是一种必败态。

同时，因为 $a_i \wedge s < a_i$ ，是合法的替换。

4.1.1.2.2 定理 2：必败态的后继状态均为必胜态

证明：

设 $a_1 \wedge \dots \wedge a_n = 0$ ，则相同位上 1 的个数为偶数。此时，无论减少哪个数，都会使得异或和 $\neq 0$ 。

必胜态与必败态交替出现，终态 $(0, 0, \dots, 0)$ 是必败态。

终态： $a_i = 0$ ，异或和为 0，必败态。 $a_i \neq 0$ ，异或和 $\neq 0$ ，必胜态。

4.1.2 台阶型 Nim 游戏

有 $1 \sim n$ 级台阶，第 i 级台阶上摆放 a_i 个石子，每次操作可将第 k 级台阶上的石子移一些到第 $k-1$ 级台阶上，移到第 0 级台阶（地面）的石子不能再移动。

如果一个人没有石子可以移动，他就输了，问先手是否必胜。

4.1.2.1 结论

若奇数台阶上的石子数异或和不为 0，即 $a_1 \oplus a_3 \oplus a_5 \oplus \dots \neq 0$ ，则先手必胜；否则先手必败。

4.1.2.2 定理

4.1.2.2.1 定理 1：必胜态的后继状态至少存在一个必败态

4.1.2.2.2 定理 2：必败态的后继状态均为必胜态

4.1.2.3 证明

情况 1：若初态奇数台阶石子异或和不为 0（必胜态），甲先手：

1. 甲选择某个奇数台阶，将其部分石子移到下一级偶数台阶，使得所有奇数台阶石子数的异或和为 0
2. 如果乙将偶数台阶的石子移到奇数台阶，甲只需将乙移动的同数量石子继续下移到下一级偶数台阶，保持奇数台阶异或和为 0
3. 如果乙将奇数台阶的石子移到偶数台阶，甲可以调整其他奇数台阶的石子，使得奇数台阶异或和重新为 0
4. 重复上述过程，最终甲将把第 1 级台阶的石子全部移到地面，获得胜利

情况 2：若初态奇数台阶石子异或和为 0（必败态），甲先手：

- 无论甲如何操作，都会破坏奇数台阶石子异或和为 0 的状态
- 乙总是可以恢复奇数台阶石子异或和为 0 的状态
- 最终乙将获得胜利

4.1.3 有向图游戏

给定一个有向无环图，图中只有一个起点，在起点上放一个棋子，两个玩家轮流沿着有向边推动棋子，每次走一步，不能走的玩家失败。

4.1.3.1 mex 运算 (minimum exclusion)

$[S]$ 为不属于集合 S 中的最小非负整数，

$$\text{mex}(S) = \min\{x \mid (x \in N, x \notin S)\}$$

例如： $-[\{0, 1, 2\}] = 3$ ， $-[\{1, 2\}] = 0$

4.1.3.2 SG 函数

设状态（节点） x 有 k 个后继状态（子节点） $y_1, y_2, \dots, y_k$,

$$SG(x) = \text{mex}(\{SG(y_1), SG(y_2), \dots, SG(y_k)\})$$

4.1.3.3 SG 定理

由 n 个有向图游戏组成的组合游戏，设起点分别为 $s_1, s_2, \dots, s_n$ ，当

$$SG(s_1) \wedge SG(s_2) \wedge \dots \wedge SG(s_n) \neq 0$$

时，先手必胜；反之，先手必败。

4.1.4 SG 定理证明

4.1.4.1 定理陈述

由 n 个有向图游戏组成的组合游戏，设起点分别为 $s_1, s_2, \dots, s_n$ ，当

$$SG(s_1) \wedge SG(s_2) \wedge \dots \wedge SG(s_n) \neq 0$$

时，先手必胜；反之，先手必败。

4.1.4.2 证明

4.1.4.2.1 基本思路

证明思路类似于 Nim 游戏的证明，通过分析必胜态和必败态的转移关系。

4.1.4.2.2 定义

- 令 $X = SG(s_1) \oplus SG(s_2) \oplus \dots \oplus SG(s_n)$
- 终态：所有游戏都处于无法操作的状态，此时 $X = 0$

4.1.4.2.3 证明步骤

4.1.4.2.3.1 1. 终态是必败态

当所有游戏都处于无法操作的状态时：- 所有 $SG(s_i) = 0$ （根据 SG 函数定义）- 因此 $X = 0 \oplus 0 \oplus \dots \oplus 0 = 0$ - 无法进行任何操作，当前玩家失败

4.1.4.2.3.2 2. 若 $X \neq 0$ ，存在操作使 $X = 0$

设 $X = k \neq 0$ ，则存在某个 $SG(s_i)$ 使得 $SG(s_i) \oplus k < SG(s_i)$

理由：- 设 k 的最高位为第 m 位 - 存在 $SG(s_i)$ 的第 m 位为 1 - 则 $SG(s_i) \oplus k$ 的第 m 位变为 0，且更高位不变 - 因此 $SG(s_i) \oplus k < SG(s_i)$

操作：- 选择该游戏 i - 将其状态从 s_i 移动到某个后继状态 t ，使得 $SG(t) = SG(s_i) \oplus k$ - 这样的 t 一定存在，因为根据 SG 函数定义， $SG(s_i)$ 是其后继状态 SG 值的 mex - 这意味着 0 到 $SG(s_i) - 1$ 的所有值都会出现在后继状态的 SG 值中

结果：- 操作后，新的异或和为：

$$\begin{aligned} X' &= SG(s_1) \oplus \dots \oplus SG(t) \oplus \dots \oplus SG(s_n) \\ &= SG(s_1) \oplus \dots \oplus (SG(s_i) \oplus k) \oplus \dots \oplus SG(s_n) \\ &= (SG(s_1) \oplus \dots \oplus SG(s_i) \oplus \dots \oplus SG(s_n)) \oplus k \\ &= k \oplus k = 0 \end{aligned}$$

4.1.4.2.3.3 3. 若 $X = 0$ ，任何操作都会使 $X \neq 0$

假设当前 $X = 0$ ，玩家选择游戏 i 并将其状态从 s_i 移动到 t

分析：- 如果 $SG(t) = SG(s_i)$ ，则异或和不变，仍为 0 - 但根据 SG 函数定义， $SG(s_i)$ 是其后继状态 SG 值的 mex - 这意味着 $SG(s_i)$ 不会出现在后继状态的 SG 值中 - 因此 $SG(t) \neq SG(s_i)$

结果：- 操作后，新的异或和为：

$$\begin{aligned} X' &= SG(s_1) \oplus \dots \oplus SG(t) \oplus \dots \oplus SG(s_n) \\ &= (SG(s_1) \oplus \dots \oplus SG(s_i) \oplus \dots \oplus SG(s_n)) \oplus SG(s_i) \oplus SG(t) \\ &= 0 \oplus SG(s_i) \oplus SG(t) \\ &= SG(s_i) \oplus SG(t) \neq 0 \end{aligned}$$

4.1.4.2.4 结论

- 从 $X \neq 0$ 的状态，玩家总可以移动到 $X = 0$ 的状态
- 从 $X = 0$ 的状态，玩家只能移动到 $X \neq 0$ 的状态
- 终态 $X = 0$ 是必败态

因此，当且仅当 $X \neq 0$ 时，先手必胜。

证毕。

5 通用 Trick 与杂项

5.1 其他/Trick/杂项相关 trick

括号序列合法的充要是 +1 -1 trick-> 所有前缀和 ≥ 0 sum=0 ->能推导出 这样的串里一定存在() 子串

考虑 m 是一个奇数长度序列中位数的充要条件是 $\geq m$ 的数 $> < m$ 的数 && $\leq m$ 的数 $> < m$ 的数 于是我们可以考虑使用 +1 -1 trick 然后算前缀和

更一般的 考虑中位数是下中位数 满足 $\geq m$ 的数 $> < m$ 的数 && $\leq m$ 的数 $\geq m$ 的数 $\geq < m$ 的数 && $\leq m$ 的数 $> < m$ 的数

sqrt 对 longlong 精度不够 要用 sqrtl

某些区间问题 可以利用前后缀处理的思想 <https://ac.nowcoder.com/acm/contest/view-submission?submissionId=79677851&returnHomeType=1&uid=719203876>

某些题可以转化成二进制来做 <https://ac.nowcoder.com/acm/contest/view-submission?submissionId=79675702&returnHomeType=1&uid=719203876> eg $x \leftarrow \text{floor}(x/2) = x >> 1$

位运算计数某些经典技巧是拆位, 每位独立贡献 <https://ac.nowcoder.com/acm/contest/view-submission?submissionId=79091623&returnHomeType=1&uid=719203876>

压位 ull st=((ull)a<<32)|b;

$x-y \leq x^{\wedge}y \leq x+y$

$$|S \oplus M| = |S \cup M| - |S \cap M|$$

S,M 是状压的集合

实际上 一个 x 异或上一个区间得到的数构成的连续段是 $\log$ 的 具体来说 考虑 $\log$ 个相同前缀的区间 e.g $11001=[00000,01111]+[10000,10111]+[11000,11001]$ 考虑他的一个后缀是任意的, 所有这个连续段的上下界可以 $O(1)$ 算

对一个 x 异或一个区间, 得到的上下界可以 $\log$ 算, 参见 2026 校赛 ex 题

对一个 x 异或一个区间, 得到的连续段是一个的充要条件好像很深刻, 我看不懂现在。

如果你要算一个数对集合内数的 $\text{xor} \leq w$, 你可以对集合建 01tire 然后在 01tire 上跑 dfs

xorhash 通常存在于某些 2/1 性质 奇偶性质的题里面 括号序列!?!?

有些题有奇偶不变量/其他的不变量

压维转化 0-base $xm+y$ 1base $(x-1)m+(y-1)+1$

曼哈顿距离 $\text{dis}(x1,y1,x2,y2)=\text{abs}(x1-x2)+\text{abs}(y1-y2)$ 切比雪夫距离 $\text{dis}(x1,y1,x2,y2)=\max(\text{abs}(x1-x2),\text{abs}(y1-y2))$ 互转 $(x,y) \rightarrow ((x+y)/2,(x-y)/2)$ 原坐标的曼哈顿距离等于新坐标的切比雪夫距离

$\max(a,b)=(a+b+\text{abs}(a-b))/2$ $\min(a,b)=(a+b-\text{abs}(a-b))/2$

dilworth 定理 dilworth 定理: 一个有向无环图的最小链划分等于其最大反链大小 通常考察一个二维偏序, 即按照一维排序后, 可以转 lis 做

hall 定理 一个二分图是完美匹配的充要条件是, 对于任意一个 X 的子集, 其邻居数大于等于该子集大小

排序不等式 $\sum a_i b_i$ 最大的充要条件为 即对于所有 i,j , 若 $a_i < a_j$ 则 $b_i < b_j$ 即 a 和 b 是同序的。最小的充要条件为 即对于所有 i,j , 若 $a_i < a_j$ 则 $b_i > b_j$ 即 a 和 b 是反序的。同理 $\max(a_i - b_i)$ 最小 $\rightarrow a$ 和 b 是同序的。 https://www.luogu.com.cn/problem/AT_arc201_d

注意 $\%m$ 的形式有时候可以破坏为链

排列诸结论 1.交换两个数的顺序, 会使排列的逆序对数 ± 1 奇偶性变换 2.一个长度为 n 的排列, 其逆序对数的奇偶性等于 $n - c$ c 是置换环数 置换环: 下标 $[l,r]$ 包含了 $[l,r]$ 的所有元素, 称为置换环 3.交换大小为 x, y 的两个块 对逆序对的贡献为 $x*y-2k$, k 为原来块间的逆序对数 4.最优交换次数 K : 将排列 P 分解为若干个不相交的置换环。设环的个数为 C , 则将 P 排序为 $(1, 2, \dots, N)$ 的最小交换次数为:

$$K = N - C$$

5.置换环内交换: 如果你交换同一个环内的两个元素, 这个环一定会分裂成两个更小的环。 6.置换环间交换: 如果你交换不同环的两个元素, 这两个环会合并成一个大环。

恰好的方案数难求的时候 考虑至少的方案数然后转化 经典的 求 $=k$ 的方案数 $= \text{求} \geq k$ 的方案数 $- \text{求} > k$ 的方案数 经典的 $k = \sum_{i \leq k} \{1\}$ 然后我们能把一个东西变成对每个 $\geq k$ 都计算一遍 式子很像的时候考虑求出整体贡献然后对每个 i 取消贡献

有时候建出操作树可以降维打击!

二分图染色可以使用拓展域并查集 具体来说是拆点 $x \rightarrow x, x+n$ x 代表黑色 $x+n$ 代表白色 (u,v) 连边的时候 $\text{merge}(u,v+n)$ $\text{merge}(u+n,v)$ 如果 $\text{same}(u,v)$ 则说明两个点同色 不成立

float 有效位数是 6-7 位 double 有效位数是 15-16 位 long double 有效位数是 18-19 位

记得 sqrtl 和 atanl $\text{acos}(-1)=\pi$

完全图最小生成树可以用 boruvka 算法 也可能可以用数据结构优化 prim 算法

树直径更新：考虑原树直径 (u,v) ，考虑树上新增的点，直径的更新只可能是 (u,new) 、 (v,new) 或者 (u,v)

$x \rightarrow \text{floor}(x/2), \text{ceil}(x/2)$ 在递归的每一层都是 $v, v+1$

经常忘的：根号做法，调和级数 根号做法 分块，定期重构，根号分治

安全取整 注意负数下取整是往负无穷，上取整是往 0 // $\text{floor}(a/b)$ `long long floor_div(long long a, long long b){ return a/b - ((a^b)<0 && a%b); }`

$\text{ceil}(a/b)$ `long long ceil_div(long long a, long long b){ return a/b + ((a^b)>0 && a%b); }` 正数 $\text{ceil } a+b-1/b$

一个经典转化是一个升序列表 $x_1, x_2, x_3 \cdots x_n$ $y_1 = x_1 - 1, y_2 = x_2 - 2$ 注意到 y_i 也是单增的。

6 字符串

6.1 字符串/笔记/一些比较神秘的 hash 手法

以下的 *Hash* 均采用这个生成：

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count()^(ull)(new char));
vector<ull> h(n+1);
for(int i=1;i<=n;i++) h[i]=Xhash::splitmix64(rng())|((ull)<<50)+1;
```

XHash 类见防 hack 的 `umap, gp_hash_table.cpp` 对字符集/数集中的每个元素都随机分配一个 *Hash* 值（大奇数）。采取自然溢出 *Hash*。

给定数组/字符串。##### 1.1.考虑求解每个元素的出现次数为偶数的子序列/子串个数。老生常谈，维护前缀异或和就行，区间 $[l, r]$ 合法的条件是 $sum[r] \oplus sum[l-1] = 0$ ，即 $sum[r] = sum[l-1]$ 。

如果他要你求区间能重排成回文串，做一些适当转化可以变成这个问题。

6.1.0.0.0.1 1.2.更一般的，考虑求解每个元素的出现次数为 k 的倍数的子序列/子串个数。

做一个前缀 *Hash* 和，维护一个 *cnt* 数组，当数组中的某个元素达到 k 时候，在前缀和上减去 $(k-1)*h[i]$ ，这样每个前缀的意义其实是一个长度为 n 的状态向量，每个位置是模 k 的。区间 $[l, r]$ 合法的条件是 $sum[r] = sum[l-1]$ 。实现：

```
for(int i=0;i<26;i++) w[i]=rng()|1;
for(int i=1;i<=n;i++){
    cnt[s[i]-'a']++;
    if(cnt[s[i]-'a']==k){
        ph[i]=ph[i-1]-w[s[i]-'a']*(k-1);
        cnt[s[i]-'a']=0;
    }
    else ph[i]=ph[i-1]+w[s[i]-'a'];
}
```

例题：<https://codeforces.com/gym/105911/problem/E>

6.1.0.0.0.2 1.3.更特殊的，考虑求解每个元素的出现次数为 k 的子序列/子串个数。

注意到我们维护一个滑动窗口，窗口里面的元素个数 $\leq k$ ，然后问题就转化成 1.2 了。例题：https://atcoder.jp/contests/abc455/tasks/abc455_g 子问题 1

6.1.0.0.0.3 2.1.考虑求解字母个数相同的子序列/子串个数。

对我们的 *Hash* 做一个小小的修改，对前 $n-1$ 个元素，保持不变，最后一个元素取前 $n-1$ 个元素哈希值和的相反数，做前缀 *Hash*，区间 $[l, r]$ 合法的条件是 $sum[r] = sum[l-1]$ 。例题：https://atcoder.jp/contests/abc455/tasks/abc455_e

6.1.0.0.0.4 3.1 考虑判断一个子区间能够进行重排后变成另一个子区间

那么其实就是我们普通的前缀 *Hash*，判断 $[l1, r1]$ 和 $[l2, r2]$ 是否相等，即 $sum[r1] - sum[l1-1] = sum[r2] - sum[l2-1]$ 。

以上的 *Hash* 概率都是没有问题的，防 *hack* 也是对的。

6.2 字符串/笔记/字符串 trick

二分 hash 能做的 1.求 lcp 长度 2.比较字符串 3.找回文串的所有出现 考虑用 # 拼接串，然后枚举中心点，二分长度，hash 判断反串和正串即可

kmp 自动机 你可以把他当作一个包含单串所有前缀的 *acam*，其实也是一个单串 *acam*，只需要在他的所有前缀打上存在标记即可

border：一个串的子串是他的真前缀和真后缀